Scope and architecture

Objective

After entry, closing every client must leave SL/target management operating. Reopening reads recorded results without restarting monitoring. Restarts restore unresolved positions without enabling new entries. Confirmed execution determines completion.

Status and limits

Planning document dated 2 October 2026. Based on the source audit in this conversation; no trading implementation has been performed. Deployment topology, hosting tier and instrument-specific broker capabilities remain unverified. No guarantee of exact stop-price execution or exactly-once broker execution is implied.

Preserve existing workflows

Keep strategy signals, indicators, entry conditions, configured SL/target points, quantity selection, cooldown rules, authentication, account-switch restrictions, charts, navigation, history filters and historical P&L unchanged. Use additive database fields and backward-compatible API additions. Paper execution correction is a separate optional release.

Required control changes

Stop Strategy stops new entries while an existing position remains managed. Uncertain entry/exit submissions block conflicting orders until reconciliation. Broker disconnect must explain which positions lose application-managed monitoring. Refresh and application logout must never start or stop a position manager.

Single-user ownership decision

Verify Node replica count, autoscaling, rolling deployment overlap, local processes and manual recovery tools first. If exactly one trading process is enforced, use per-position in-process locks plus atomic durable transitions. A permanent owned flag is unsafe after a crash. If multiple trading runtimes can overlap, introduce expiring MongoDB leases and ownership checks. Add fencing/version controls only when justified; leases cannot make a remote broker operation exactly-once.

Extend Phase D

Retain existing protective-stop placement, POSITION_UNPROTECTED behavior and urgent alerts. Extend working-status verification, cancellation/fill reconciliation and recovery. Do not create a second protection implementation. Moving lifecycle ownership out of strategyEngine.js is an extraction, with existing interfaces retained where possible.

Priority and delivery gates

P0-A: durable execution coordination

Closes Critical duplicate-sell and invisible-entry gaps. Persist entry and exit intent, exact contract, attempts and broker IDs; reconcile ambiguous submissions before retrying. Depends on additive schema and baseline fixtures.

P0-B: safe stop/exit coordination

Closes Critical protective-stop cancellation races. Verify final cancellation and intervening fills before target/manual sells or stop replacement. Use remaining quantity, not requested entry quantity. Depends on the execution coordinator.

P0-C: independent management and recovery

Closes Critical monitoring destruction and restart loss. Persist exit settings/trailing state, separate scanner from position owner, discover all unresolved records after database readiness and periodically. Depends on P0-A/B; restoring unsafe exits automatically is not acceptable.

P0-D: authoritative completion

Closes High inaccurate/lost-result findings. Save verified fills, quantities and average prices; retry persistence idempotently. Remove synthetic zero-P&L cleanup and rejection-text-based closure. Never discard unresolved state after a database failure.

P1: protection, availability and session resilience

Extend protection lifecycle handling, enforce fresh quotes and bounded non-overlapping requests, recover sessions where supported, and reconcile next-session DAY-order expiry. Hosting availability, usable authentication and fresh data are prerequisites for unattended operation. Elaborate coalescing or distributed infrastructure can wait; minimum reliability controls cannot.

P2: presentation and paper correction

Expose durable state through existing pages and APIs. Deliver fresh-quote paper entry correction separately because future paper fills/P&L intentionally change. Preserve existing history.

Release gates

Ship small reviewable changes. Gate 1: additive schema and diagnostics. Gate 2: read-only reconciliation observation with no order authority. Gate 3: PAPER recovery and fault tests. Gate 4: verified LIVE broker adapter and execution coordination in staging. Gate 5: narrowly enabled LIVE rollout. Gate 6: separate paper-price change. Never run old and new execution paths as simultaneous owners.

Persistence and state transitions

Trade.js: additive fields

Persist broker account identity, exchange, token, symbol, expiry, product and side; requested/filled/remaining quantity; entry intent and broker order ID; actual average entry price; immutable entry-time exit configuration; resolved SL/target; trailing level/revision; protection IDs/status/quantity/validity; exit intent/reason/time; reconciliation status; quote health/timestamps; owner/heartbeat and lease fields only if required.

Execution attempts

Use a durable attempt collection or embedded bounded attempt records with trade ID, operation kind, intent ID, requested quantity, payload fingerprint, submission state, broker order ID, acknowledgement time, last reconciliation and error. Index unresolved records and intent IDs. Preserve uncertain attempts; do not overwrite an unknown result with a fresh attempt.

Separate state dimensions

Position lifecycle: ENTRY_PENDING, OPEN, EXIT_PENDING, CLOSED, RECONCILIATION_REQUIRED. Protection: NOT_APPLICABLE, PENDING, WORKING, PARTIAL, UNPROTECTED, EXPIRED, UNKNOWN. Monitoring: ACTIVE, DEGRADED, AUTH_REQUIRED, STOPPED. Keep legacy positionState mapped for old consumers; PAPER must not imply real broker protection.

Atomic transition rules

Use conditional updates from the expected state and revision. Persist intent before network submission. Mark SUBMITTING before the call; a crash here requires reconciliation, not blind resubmission. Persist acknowledgement and fills afterward. CLOSED requires zero verified remaining quantity and adequate execution evidence, or explicit unresolved handling when evidence is unavailable.

Historical and legacy records

Do not rewrite completed trades or historical P&L. Classify every unresolved legacy record. Recover only with sufficient exact contract and original settings; otherwise flag reconciliation and show the missing fields. Current strategy settings are not substitutes for historical settings. Retain identifiers and protective orders during migration.

Database failure policy

Before a new entry: fail closed if intent cannot be persisted. After broker submission: retain uncertain state and alert, reconcile when persistence returns. On exit completion-save failure: retain/retry durable recovery work and do not resume entry scanning as if completion were fully recorded. Verify write acknowledgement for critical transitions.

Entry, exit and protection execution

Entry sequence

Persist exact contract/configuration and entry intent; acquire lock/ownership; submit once; persist acknowledgement; reconcile delayed/partial fills; manage confirmed filled quantity promptly; verify supported broker protection. A BUY response timeout or missing acknowledgement remains unresolved. Do not silently return to scanning or submit another BUY.

Exit sequence

Acquire per-position lock and verify ownership. Persist exit intent and reason. Reconcile current position and all related attempts/protection. Confirm protective cancellation and account for intervening fills. Submit only verified remaining quantity. Persist broker ID and follow updates/polling to terminal result. Reconcile unknown outcomes before any retry.

Partial fills and concurrent triggers

Use cumulative fills deduplicated by broker execution identity where available. Update remaining quantity atomically. A target, manual exit, stop fill and trailing update all enter one coordinator. Repeated callbacks must be idempotent. Never sell full original quantity after a partial exit. Insufficient-quantity rejection is evidence to investigate, not proof of completion.

Phase D protection extension

After entry, confirm protection is working; an ID alone is insufficient. Monitor pending, rejected, cancelled, expired, partial, complete and missing orders through updates plus periodic reconciliation. Persist unprotected state and send the existing urgent alert when protection cannot be established. Deduplicate repeated alerts while retaining meaningful state changes.

Trailing stop replacement

Prefer verified broker amendment where supported. Otherwise reconcile old protection, cancel and confirm terminal state, recompute remaining quantity, submit replacement and verify it. Persist any temporary gap. If the old stop fills during replacement, do not place another full-quantity sell. Keep configured trailing calculations unchanged.

Broker capability gate

For each exchange/product, verify supported stop order types, required variety, trigger/limit fields, price ticks, quantity rules, validity, order status meanings and cancellation semantics against current official broker documentation and staging responses. Verify paired exits before using them. Two unrelated sells are not a safe OCO mechanism. No live validation orders without explicit authorization.

Recovery, quotes and sessions

Independent position manager

Extract position lifecycle from SingleStrategyRunner into positionManagementService.js; centralize broker reconciliation in brokerReconciliationService.js and submission coordination in executionCoordinator.js. Strategy runners retain signal generation and entry scanning. These are proposed modules, not a parallel rewrite of existing broker adapters.

Startup recovery

After MongoDB readiness, discover all unresolved entries and positions across users. Acquire ownership, restore PAPER settings, reconcile LIVE account/positions/orders/fills, restore the original token/expiry, and finalize only verified completion. Missing account/session/settings retain explicit reconciliation or authentication-required state. Recovery never enables new entries.

Periodic discovery and subscriptions

Repeat discovery to recover records skipped during outages and stalled intents. Separate scanner and position subscription reference counts. Stopping scanner listeners must retain position price subscriptions. Expose owner age, unresolved count and degraded count through health diagnostics. Shutdown stops new entries and leaves durable state; it does not automatically liquidate.

Minimum quote reliability

Use exchange+token identity, finite positive prices and trustworthy timestamps. Reject stale/mismatched data; explicitly handle absent timestamps. Prevent overlapping position polls; use bounded request timeouts, backoff and 429 handling. Prefer existing validated streams with bounded REST fallback. Add shared coalescing only if measured load requires it. Persist triggered exit intent and trailing changes before relying on them.

Session and next-day recovery

Extend broker_session.py with explicit authentication states and verified supported refresh behavior. Store only permitted session material securely if restart restoration requires it; do not silently store MPIN/TOTP. Restore market/order streams, then reconcile before exits resume. If authentication cannot recover, report the limitation and request reconnection.

Trading-day and hosting prerequisites

Verify Node and Python hosting do not idle-suspend and that the singleton deployment assumption holds. Health checks cannot replace persistence. Reconcile DAY protection at the next session, holidays, contract expiry and manual broker exits. Do not assume overnight stop survival. Following data outages, use fresh data and broker fills; do not invent a historical threshold execution.

Frontend compatibility and regression protection

Authoritative read paths

Extend strategy status with persisted position/protection/monitoring state and latest recorded completion while retaining existing response fields. Keep scanning status separate. Status reads and page mounts must never start strategies, acquire execution ownership or submit orders. History continues using recorded trades.

Controls and messaging

Keep current navigation and requests where possible. Relabel/describe Stop as stopping new entries while managing an open position. Manual Exit routes through the coordinator. Broker disconnect shows affected positions and authentication limitations. Preserve account-switch and active-trade deletion restrictions; do not weaken them as runtime state moves.

Separate paper correction

Retain Heiken Ashi signal price as diagnostic input and use a fresh execution quote for new PAPER entries. Keep signal generation and configured exit rules intact. Preserve historical records. Release independently with fixtures demonstrating the expected changed execution price and unchanged signal.

Unchanged strategy fixture suite

Before extraction, capture deterministic inputs and outputs for every strategy: indicator values, entry signals, side/contract selection, SL/target and trailing arithmetic, configured quantity, entry suppression and cooldown. Compare after refactoring. Any intentional difference requires separate scope and approval.

API and UI regression suite

Verify login/register/refresh/logout, broker connect/disconnect, account switching, strategy start/stop, manual exit, status polling, charts, history filters/pagination/stats, open-trade queries and deletion guards. Check legacy documents and old response consumers. Use meaningful behavior tests, not assertions that merely mirror implementation.

Integration boundaries

Reuse existing requestPython, marketDataService, orderUpdateService and notificationService wherever their behavior is adequate. Add explicit timeouts at trading call sites rather than changing unrelated calls globally without coverage. Maintain PAPER/LIVE isolation and correct broker-account routing. No new infrastructure or credentials unless a verified requirement demands them.

Fault-injection acceptance matrix

Client independence

PAPER entry at 100, all clients close, fresh price crosses configured 90 SL, position completes and reopening shows recorded result without Start. Repeat for target. App logout and Stop Strategy leave existing position managed; Stop suppresses new entries.

Restart and authentication

Restart Node before/after entry acknowledgement, during protection placement, during exit submission and after fill before save. Restart Python and expire sessions. Expected: durable discovery, no automatic new entries, reconciliation before order authority, explicit authentication-required state when recovery is unavailable.

Ambiguous orders and partial fills

Simulate accepted BUY/SELL with lost response, delayed fills, missing orderbook result, 5-second confirmation timeout, partial entry/exit, late WebSocket updates and duplicated events. Expected: no blind resubmission, remaining quantity tracked, uncertain attempts visible and completion based on evidence.

Stop/target and trailing races

Trigger stop fill during target cancellation, manual exit during target exit and fill during stop replacement. Expected: one coordinator, cancellation/fill reconciliation, no knowingly excessive sell quantity and clear protection-gap alert.

Quotes and database outages

Inject stale/string/NaN/mismatched quotes, missing timestamps, stream disconnect, REST timeout and 429; fail DB before intent, after submission and after fill. Expected: no stale-price decisions, bounded polls, durable intent, retryable completion and no silent position loss.

Ownership and overnight

If singleton: demonstrate overlapping trading processes cannot start during deployment. If leases: test lease expiry, lost renewal and competing recovery workers. Test DAY expiry, holidays, expired contract and manual broker close. Expected: safe owner handover or explicit unresolved state; no guessed contract/configuration or zero-P&L cleanup.

Validation method

Use fake broker adapters, controlled clocks and isolated database fixtures first; never run fault tests against the real account. Then staging PAPER end-to-end checks. Broker capability validation precedes any authorized narrow LIVE trial. Keep evidence with each release gate: scenario, expected state, actual state, order count and durable records.

Rollout, rollback and completion

Operational controls

Feature flags separate diagnostics, recovery ownership, new execution coordinator and paper-price correction. Capture account/position/intent/order IDs in structured logs with credentials redacted. Alert on unprotected positions, ambiguous submissions, absent owner, prolonged quote degradation, failed completion writes and expired protection.

Safe rollout sequence

Deploy schema/indexes and baseline tests first. Observe reconciliation without mutation. Enable new ownership for isolated PAPER trades; validate restart/fault scenarios. Enable LIVE only after capability and cancellation semantics are verified and existing open positions are classified. Switch one execution path at a time; document operator checks before widening scope.

Rollback

Disable new entry scanning first when execution safety is uncertain. Preserve broker protective orders and all additive state/attempt records. Continue a compatible recovery manager for already-migrated positions or place them in explicit operator reconciliation. Do not downgrade to an older runtime that ignores new attempts or creates duplicate owners. Do not drop fields or cancel protection as a rollback shortcut.

Completion checklist

Every unresolved position has a valid owner or explicit reconciliation/authentication-required state. Client closure and application logout do not affect management. Stop halts entries only. Restart restores management without scanning. No ambiguous submission is blindly retried. Remaining quantities and verified fills determine closure. Completion survives database recovery. History remains readable. Regression and fault gates pass with retained evidence.

Outstanding decisions before implementation

Confirm one active Node trading process and deployment overlap; establish hosted service availability; verify broker capabilities per instrument/product; choose safe permitted session restoration; classify existing open trades; select singleton enforcement or lease mode. These are targeted discovery tasks, not reasons to introduce a multi-tenant architecture by default.

Source traceability

Audit anchors: backend/models/Trade.js; backend/index.js; backend/services/strategyEngine.js (recovery, monitor, protective placement/replacement, executeExit); backend/services/marketDataService.js; backend/services/orderUpdateService.js; backend/controllers/authController.js; backend/controllers/tradeController.js; angel-one/app/services/broker_session.py; angel-one/app/main.py; strategy-page polling cleanup. This plan extends the supplied Phase D implementation and the revised single-user scope.